

PREPRINT / ESTUDIO EXPLORATORIO

Coordinación de subagentes: SAGE-CREH y selección de skills

Cristhian Egoavil / 13 de septiembre de 2026 / v2.0

Resumen

Proponemos SAGE-CREH, una quinta arquitectura que combina selección de skills, vía directa, resolutores independientes y arbitraje condicional. Una revisión focalizada de doce papers de arXiv fundamenta el diseño. Tras 20 ejecuciones de desarrollo separadas, se realizaron 216 ejecuciones reales con el mismo modelo y las doce tareas originales, dos repeticiones y nueve condiciones: cuatro arquitecturas, un agente único, SAGE y tres controles de ablación. La réplica consumió 707,090 tokens en 510 llamadas. SAGE obtuvo 23/24 aciertos estrictos y 23/24 tras normalizar únicamente el envoltorio SQL, con 1,765 tokens y 7.34 segundos medios por tarea. Se separan coordinación, carga de instrucciones y formato de salida mediante controles explícitos. La muestra es pequeña, sintética y conocida durante el diseño; la comparación es una réplica informada por desarrollo, no evidencia de superioridad universal ni de generalización a ciegas.

Palabras clave: subagentes; delegación adaptativa; skills; tokens; tiempo; evaluación emparejada.

Código y datos: <https://github.com/CristhianRudolf7/sage-creh>

1. Pregunta de investigación

La unidad de diseño estudiada no es el modelo, sino la organización de sus llamadas: quién resuelve, quién revisa, cuándo se delega y qué instrucciones recibe cada rol. Manteniendo el modelo fijo, se pregunta si una quinta arquitectura adaptativa puede conservar el cumplimiento de las cuatro arquitecturas originales con menos coordinación. Se distinguen tres objetivos: resolver la tarea completa, consumir menos tokens agregados y reducir la espera del usuario. Ninguno sustituye a los otros.

La versión 1 comparó flujos secuencial, paralelo, jerárquico y reflexivo con un agente único y una ablación de skills. La versión 2 implementa SAGE-CREH, evalúa dos políticas en desarrollo y vuelve a ejecutar todos los controles. La contribución es esta implementación acotada con evidencia reproducible, no una prueba de superioridad universal ni la invención de la delegación selectiva.

2. Fundamento documental: nueve referencias originales

La revisión incluye nueve papers enviados inicialmente a arXiv entre febrero y septiembre de 2026. Se comprobaron las fechas de envío por separado de las revisiones; las versiones consultadas figuran en las referencias. La selección se centró en coordinación, traspaso de contexto, asignación de skills y eficiencia medible. Es una revisión técnica focalizada, no una revisión sistemática exhaustiva. Sus puntuaciones no se combinan con el experimento posterior.

2.1. Topología de ejecución y autoridad

Kulkarni y Kulkarni comparan flujos secuenciales, paralelos, supervisor–trabajador y reflexivos para extraer documentos financieros. Sus descripciones motivan las cuatro familias estudiadas aquí. El dominio y la configuración de modelos/precios de enero de 2025 limitan la extrapolación a un asistente personal; no se reproducen sus configuraciones ni su rendimiento publicado.¹

DESBench estudia coordinación centralizada, jerárquica, heterárquica y holónica en planificación industrial. Su heterarquía implementada utiliza pujas mediadas. Amplía la pregunta desde la forma del grafo hasta quién puede asignar trabajo, rechazar compromisos y resolver conflictos; su simulador industrial constituye otro entorno de evaluación.²

ParaGUI envía trabajadores de interfaz gráfica en paralelo por rondas e integra sus resúmenes. Aporta evidencia sobre trabajo interactivo divisible e identifica la ejecución del trabajador como una limitación. Sus controles del historial visual y el entorno de varios escritorios impiden trasladar sus cifras directamente a este experimento textual.³

2.2. Traspaso de información y evaluación

OrchBench representa dependencias y transferencias entre agentes y simula calidad, tiempo de terminación y tokens. Su mecanismo de retención convierte la pérdida de información en una variable de coordinación. Aquí se ejecutan todos los trabajadores contra la API; no se sustituyen consumo o respuestas observados por resultados o costes simulados.⁴

CRAFT entrega vistas parciales complementarias de una construcción. Distingue razonamiento individual y comunicación efectiva y describe correcciones repetidas improductivas. Motiva conservar evidencia y evaluar el resultado final. Como todos nuestros trabajadores reciben la tarea original, no se estudia su problema de observación parcial.⁵

2.3. Skills y contexto específico

Subagents vs Agent Skills distingue paquetes procedurales con interfaces explícitas de entrada/salida y texto poco estructurado. Su evaluación principal utiliza un subconjunto seleccionado de 64 tareas e informa menor contexto máximo con mayor consumo total. Esto respalda estudiar la carga de información por separado de la topología.⁶

EvoAgent combina delegación jerárquica y selección por palabras clave, embeddings y modelo. Su evaluación de dominio utiliza jueces modelo; esas puntuaciones no establecen fiabilidad de ejecución exacta. Se toma como referencia la separación de recuperación y delegación, sin implementar aprendizaje autónomo ni reproducir su cascada de selección.⁷

Swarm Skills empaqueta roles, flujos, límites y dependencias como recursos reutilizables de coordinación. Su caso cualitativo deja pendiente validar ampliamente la compatibilidad. Se consideran las descripciones portables una propuesta de diseño, no evidencia de que cualquier

¹Kulkarni y Kulkarni, arXiv:2603.22651v1, 2026.

²Wang et al., arXiv:2605.13172v1, 2026.

³Yu et al., arXiv:2607.22689v1, 2026.

⁴Ren et al., arXiv:2607.25656v1, 2026.

⁵Nath, VanderHoeven y Krishnaswamy, arXiv:2603.25268v2, 2026.

⁶Piriyakulkij et al., arXiv:2609.09233v1, 2026.

⁷A. Zhang et al., arXiv:2604.20133v3, 2026.

framework multiagente pueda intercambiarlas sin comprobar su funcionamiento.⁸

Xu y Yan revisan la carga progresiva: descubrir metadatos, cargar instrucciones elegidas y recuperar recursos de apoyo. Esto distingue el catálogo del contenido insertado en cada contexto. Como revisión, orienta la política de carga, pero no aporta un resultado de rendimiento independiente para nuestras arquitecturas.⁹

2.4. Tres referencias adicionales para la arquitectura adaptativa

Uno-Orchestra combina descomposición selectiva y delegación mediante una política aprendida. Inspira decidir cuándo merece la pena delegar; aquí no se reproduce su entrenamiento.¹⁰ ProgRouter orienta decisiones de ejecución por progreso, tiempo y coste. Nuestra compuerta no es su predictor aprendido ni selecciona entre modelos.¹¹ TROVE conserva continuaciones válidas y modifica rutas ante evidencia intermedia. Aquí se adopta una revisión condicional simple, sin su grafo de skills aprendido offline.¹² La revisión ampliada reúne doce papers de arXiv; se mantienen las versiones identificadas, sin combinar sus puntuaciones con este experimento.

3. Las cinco arquitecturas y la referencia individual

Todos los agentes son llamadas sin estado al mismo modelo. Un rol distinto no significa pesos entrenados distintos. Los traspasos contienen respuestas visibles, hallazgos o dudas, nunca razonamiento privado. Cada llamada recibe la tarea pública completa; no se evalúa partición de observaciones ni contexto de producción reducido.

Patrón	Implementación	Llamadas
Individual	Un resolutor entrega la respuesta.	1
Secuencial	Extracción de evidencia, solución e integración, en tres etapas.	3
Paralelo	Dos especialistas complementarios simultáneos y un integrador.	3
Jerárquico	Supervisor planifica uno o dos encargos, trabajadores simultáneos, integración.	3–4
Reflexivo	Propuesta y crítica; revisión si hay objeciones, máximo dos rondas.	2–5
SAGE-CREH	Vía directa o dos resolutores; revisión solo por señales definidas.	1–3

3.1. SAGE-CREH: ejecución con selección de skills y compuertas

Un coordinador determinista ligero enruta por la estructura pública de la tarea. No utiliza una llamada de planificación previa. Selecciona un skill y decide si corresponde una sola propuesta o dos resolutores completos simultáneos. El segundo prioriza restricciones, dependencias y desempates; no ve la primera propuesta. Esta redundancia adaptativa difiere del reparto complementario fijo del patrón paralelo original.

1. Identificar el contrato público y cargar únicamente sus instrucciones pertinentes.
2. Marcar riesgo estructural si hay al menos ocho nodos y algún nodo con dos o más predecesores;

⁸X. Zhang et al., arXiv:2605.10052v2, 2026.

⁹Xu y Yan, arXiv:2602.12430v4, 2026.

¹⁰Cui et al., 2026.

¹¹Li, Abdelmoniem y Wang, 2026.

¹²T. Wang et al., 2026.

o al menos diez elementos de selección con incompatibilidades y prerequisites. Son reglas previas fijas, no umbrales aprendidos durante evaluación.

3. Sin riesgo, pedir una propuesta. Si cumple el esquema y no declara dudas concretas, terminar. En caso contrario, pedir una revisión final.
4. Con riesgo, pedir dos propuestas independientes a la vez. Si ambas cumplen el contrato, no declaran dudas y coinciden exactamente tras ordenar claves JSON, terminar sin integrador. Coincidir no demuestra corrección.
5. Ante discrepancia, formato inválido o dudas, un árbitro vuelve a comprobar los candidatos contra los datos originales y entrega la respuesta final.

La compuerta comprueba claves, tipos y estructura; no calcula la solución, no ejecuta SQL y no consulta referencias o fixtures. Por ejemplo, una duración entera incorrecta puede atravesarla. La confianza verbal no se usa como puntuación de verdad. El árbitro no recibe calificaciones del evaluador y puede equivocarse. No hay reintentos selectivos hasta obtener una respuesta correcta.

Si q es la fracción de tareas que activa dos resolutores, v la proporción de revisión en la vía directa y a la de arbitraje en la vía paralela, el número medio de llamadas, sin fallos de transporte, es

$$\mathbb{E}[N] = 1 + q + (1 - q)v + qa.$$

La fórmula expresa los mecanismos medidos, no una predicción de exactitud. En la vía paralela la espera de los resolutores es aproximadamente el máximo de sus duraciones, no su suma; los tokens de ambos sí se suman. Ahorrar un integrador reduce una llamada completa, pero solo es defendible si el resultado conserva el nivel de cumplimiento requerido.

3.2. Repartir roles y skills son decisiones distintas

El adaptador reconoce estas interfaces cerradas por sus campos, no por el identificador de la tarea. No mide el coste ni los errores de recuperación semántica de skills desde una conversación abierta.

Señal pública	Skill cargado	Responsabilidad
Eventos con revisión	Registros	Revisiones, estado y sumas monetarias.
Participantes	Calendario	Zonas horarias e intersección de disponibilidad.
Lista de tareas	Dependencias	Precedencias, inicios y camino crítico.
Elementos candidatos	Selección	Presupuesto, cobertura y desempates.
Eventos con secuencia y ámbito	Memoria	Confirmaciones, borrados y herencia.
Esquema SQL	SQL	Consulta de solo lectura y agregación correcta.

Todos los trabajadores de una tarea necesitan el mismo skill temático; sus roles son diferentes. No se inventan skills distintos por agente para simular diversidad. Se reutilizan sin cambios los cuerpos de las diez instrucciones originales. Los cuatro skills distractores también se conservan para las ablaciones.

4. Diseño experimental

4.1. Desarrollo separado y congelación

Antes de la réplica se probaron diez instancias nuevas: dos por cada familia no SQL, con semillas distintas de las originales. La política serial revisaba las tareas de riesgo después de una propuesta; la paralela empleaba dos resolutores simultáneos. Se decidió por mayor cumplimiento estricto, desempate por menos tokens y después menor tiempo. Los contratos SQL se comprobaron offline; no hubo piloto SQL vivo nuevo. El resultado y coste de ambos candidatos se presentan por separado.

El protocolo es local y previo a la ejecución, no un prerregistro independiente. Código, prompts, límites y orden se copiaron y hasheadon antes de cada fase. Una corrección de comparación entre tuplas Python y listas JSON se hizo antes de la primera llamada de evaluación, sin cambiar tareas ni política. Se conserva la nota de cambio. No se modificó la arquitectura durante la réplica final.

4.2. Tareas y comparación emparejada

Se repitieron exactamente los doce problemas originales y sus fixtures, dos veces por condición. Cada una de seis familias contiene dos problemas: registros con 18 eventos, agenda de tres participantes, grafos de once nodos, selección de cuatro entre doce elementos, memoria de 24 eventos y dos consultas SQL. Las diez tareas no SQL requieren coincidencia exacta; cada consulta se ejecuta sobre tres bases de prueba ocultas para los agentes, con permisos de solo lectura.

Se ejecutaron nueve condiciones: agente único; las cuatro arquitecturas originales; SAGE-CREH; jerarquía con diez skills; SAGE sin delegación; y SAGE con diez skills. Son 216 ejecuciones, 24 por condición. Se aleatorizaron bloques tarea-repetición y condiciones dentro de cada bloque, con semilla 20260914. No se ejecutaron condiciones simultáneamente; dentro de cada flujo se permitieron dos llamadas paralelas.

4.3. Controles, límites y posibles factores de confusión

Se mantuvo GPT-5.6 Luna, esfuerzo low, Responses API, JSON-object, sin streaming y máximo 1536 tokens de salida por llamada, incluido razonamiento. Timeout de conexión de 10 segundos y lectura de 60; cero reintentos de transporte; máximo general de cinco llamadas y límite de 240 segundos para iniciar otra llamada. Este último no interrumpe una petición HTTP ya iniciada.

Los controles originales conservan prompts y catálogo de diez descripciones más un cuerpo de skill seleccionado por familia. SAGE omite el catálogo y explicita el anidamiento JSON, especialmente SQL. Por tanto, compararlo con los originales cambia más que la topología. Para aislar efectos se añaden dos controles: *SAGE sin delegación*, mismo proponente y payload pero siempre una llamada; y *SAGE con diez skills*, misma política y prompts, todos los cuerpos. La pareja jerárquica original también se vuelve a ejecutar. Los presupuestos máximos por llamada son iguales, no los tokens realmente consumidos.

4.4. Medición y análisis

El cumplimiento primario exige todos los campos correctos en el sobre JSON original. Como métrica secundaria predefinida para v2, si SQL aparece como cadena directa dentro de `answer`, se envuelve esa misma cadena en el campo `sql` y se aplica el mismo evaluador. No se edita SQL ni

se corrigen respuestas no SQL. La métrica primaria nunca se repara.

Los tokens son la suma de entrada y salida informadas por todas las llamadas, incluidos supervisión, crítica, contexto repetido y fallos observados. Entrada cacheada y razonamiento ya pertenecen a esas cantidades. El tiempo monótonico incluye coordinación, red y trazas hasta la respuesta final; excluye calificación. No es la suma de las duraciones HTTP solapadas ni un benchmark de throughput.

Se estiman intervalos percentiles exploratorios del 95 % con 10000 remuestreos emparejados por tarea, manteniendo juntas ambas repeticiones y condiciones. Se informan diferencias de cumplimiento, tokens y tiempo y razones de consumo. Doce tareas distintas no equivalen a 216 tareas independientes. Los intervalos degenerados al alcanzar el techo no prueban fiabilidad perfecta.

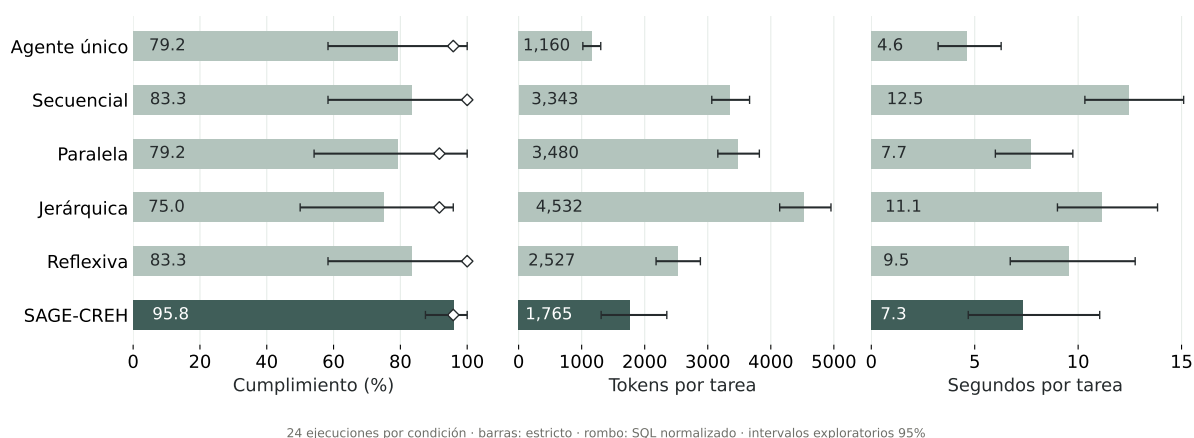
5. Resultados de la réplica v2

Las 216 ejecuciones completaron 510 llamadas y 707,090 tokens. Se registraron 0 errores de infraestructura o protocolo. La contabilidad de todas las llamadas fue completa; los tokens no se estimaron.

5.1. Comparación principal: mismas tareas y modelo

Configuración	Estricto	SQL norm.	Tokens	s/tarea	Llamadas
Agente único	19/24	23/24	1,160	4.62	24
Secuencial	20/24	24/24	3,343	12.47	72
Paralela	19/24	22/24	3,480	7.69	72
Jerárquica	18/24	22/24	4,532	11.13	96
Reflexiva	20/24	24/24	2,527	9.54	54
SAGE-CREH	23/24	23/24	1,765	7.34	38

Cada fila contiene 24 ejecuciones. Tokens y segundos son medias por tarea; llamadas es el total por condición. SQL norm. es una métrica secundaria de formato, no una reparación de respuestas.



Barras de cumplimiento estricto y marcas de SQL normalizado. Intervalos percentiles por tarea del 95 %; no prueban una clasificación general de arquitecturas.

5.2. Cumplimiento por familia de tareas

Familia	Único	Sec.	Par.	Jer.	Refl.	SAGE
Registros	4 / 4	4 / 4	4 / 4	4 / 4	4 / 4	3 / 3
Calendario	3 / 3	4 / 4	4 / 4	2 / 2	4 / 4	4 / 4
Dependencias	4 / 4	4 / 4	2 / 2	4 / 4	4 / 4	4 / 4
Selección	4 / 4	4 / 4	4 / 4	4 / 4	4 / 4	4 / 4
Memoria	4 / 4	4 / 4	4 / 4	4 / 4	4 / 4	4 / 4
SQL	0 / 4	0 / 4	1 / 4	0 / 4	0 / 4	4 / 4

Cada celda: aciertos estrictos / aciertos con SQL normalizado, ambos sobre cuatro ejecuciones de la familia (dos tareas por dos repeticiones). Fuera de SQL son la misma métrica.

5.3. Controles de skills y delegación

Configuración	Estricto	SQL norm.	Tokens	s/tarea	Llamadas
Jerárquica	18/24	22/24	4,532	11.13	96
Jerárquica + 10 skills	20/24	22/24	8,578	10.62	96
SAGE-CREH	23/24	23/24	1,765	7.34	38
SAGE sin delegación	21/24	21/24	1,110	5.93	24
SAGE + 10 skills	22/24	22/24	2,966	6.38	34

SAGE sin delegación conserva el proponente y las instrucciones de SAGE, eliminando solo el segundo resolutor y la revisión. SAGE con diez skills conserva la política, cambiando la carga de instrucciones. Los controles originales incluyen un catálogo de descripciones que SAGE no recibe; no atribuir toda diferencia a la topología.

5.4. Diferencias emparejadas de consumo y tiempo

Comparación	Cambio tokens (%)	IC 95 %	Diferencia s	IC 95 %
Agente único	+52.2	[15.9, 95.3]	+2.73	[1.20, 4.97]
Secuencial	-47.2	[-61.5, -29.5]	-5.13	[-6.29, -3.77]
Paralela	-49.3	[-62.3, -33.8]	-0.35	[-1.73, 1.52]
Jerárquica	-61.1	[-71.4, -48.5]	-3.79	[-5.25, -2.30]
Reflexiva	-30.2	[-51.8, -3.5]	-2.20	[-5.33, 0.50]
SAGE sin delegación	+59.1	[21.3, 99.0]	+1.42	[0.46, 2.58]
SAGE + 10 skills	-40.5	[-49.7, -29.6]	+0.96	[-0.49, 2.94]

En cada fila: SAGE menos la configuración indicada; el cambio porcentual de tokens es $100(SAGE/control - 1)$. Valores negativos favorecen menor consumo o tiempo de SAGE. Intervalos exploratorios de 10000 remuestreos emparejados por tarea.

5.5. Rutas ejecutadas y efecto de las revisiones

SAGE-CREH. Directa: 13; Acuerdo paralelo: 5; Arbitraje paralelo: 3; Revisión selectiva: 3. Frente al primer candidato: 5 correcciones y 0 regresiones de acierto estricto.

SAGE sin delegación. Directa: 24. Frente al primer candidato: 0 correcciones y 0 regresiones de acierto estricto.

SAGE + 10 skills. Directa: 15; Acuerdo paralelo: 7; Arbitraje paralelo: 1; Revisión selectiva: 1. Frente al primer candidato: 2 correcciones y 0 regresiones de acierto estricto.

Esta inspección usa el evaluador después de terminar las ejecuciones; nunca alimentó la política de enrutamiento. Una corrección puede ser de formato. Se conserva el detalle por ejecución.

5.6. Desarrollo: resultados excluidos de la réplica

Serial: 9/10; 1,606.7 tokens y 9.07 segundos medios.

Paralela: 10/10; 1,628.4 tokens y 8.11 segundos medios.

Se eligió la política paralela por mayor cumplimiento, según la regla previa. Desarrollo consumió 32,351 tokens en 28 llamadas y 20 ejecuciones. Este coste no se oculta en la réplica ni se suma a sus medias.

El único acierto que separó a los candidatos ocurrió en DEV22, de calendario: ambas políticas usaron la misma vía directa, sin riesgo ni revisión. La diferencia puede deberse a variación de generación y no demuestra una ventaja causal del paralelismo. Se conserva la elección conforme a la regla previa, sin reinterpretarla como prueba de superioridad.

6. Interpretación y aplicación

SAGE-CREH obtuvo el mayor cumplimiento estricto de la comparación principal: 23/24 (95.8 %), con 1765 tokens y 7.34 segundos medios. Frente a los cuatro patrones originales redujo tokens un 47.2 % respecto al secuencial, 49.3 % al paralelo, 61.1 % al jerárquico y 30.2 % al reflexivo. Son diferencias observadas; los intervalos de diferencias de acierto estricto incluyen cero. La ventaja media de tiempo frente al paralelo fue de solo 0.35 segundos, con intervalo que incluye cero; tampoco se establece una ventaja de latencia frente al reflexivo.

La conclusión depende del contrato de salida. Con SQL normalizado, secuencial y reflexivo lograron 24/24; SAGE y el agente único original, 23/24. Este último consumió 1160 tokens y 4.62 segundos medios, menos que SAGE. En el frente de Pareto de los seis diseños principales aparecen agente único y SAGE con la métrica estricta; al usar la secundaria aparecen agente único y reflexivo. Son fronteras de puntos estimados, no pruebas de dominancia poblacional. SAGE no es, por tanto, la mejor elección bajo todas las definiciones de éxito.

El control SAGE sin delegación logró 21/24 con 1110 tokens: añadir coordinación elevó el consumo un 59.1 % y produjo dos aciertos más en promedio de la muestra, sin una diferencia de cumplimiento concluyente. Sus dos fallos SQL contienen una clave extra **uncertainties** en el sobre final; esa forma no está cubierta por la normalización secundaria predefinida, que solo envuelve cadenas SQL. Por ello, tampoco esta columna separa todo posible error de formato del contenido.

La inspección de las propias trazas de SAGE identifica cinco mejoras frente a su primer candidato: tres revisiones SQL de contrato y dos arbitrajes en dependencias y selección. No hubo regresiones estrictas tras revisión. Se aceptaron trece respuestas directas y cinco acuerdos paralelos sin integrador; hubo tres arbitrajes y tres revisiones selectivas, para 38 llamadas en 24 tareas. Esto documenta mecanismos concretos, sin convertir cada corrección de contrato en razonamiento multiagente superior.

El único fallo final de SAGE ocurrió en T02, primera repetición: todas las sumas monetarias coincidían, pero se devolvieron siete eventos cuando correspondían ocho. El contrato era válido,

no se declararon dudas y la vía directa lo aceptó. Este caso muestra exactamente qué no detecta la compuerta de formato.

Seleccionar instrucciones redujo tokens un 40.5 % respecto a SAGE con diez skills (1765 frente a 2966). No redujo el tiempo observado: 7.34 frente a 6.38 segundos, con diferencia incierta. Las respuestas también cambiaron las rutas: 38 frente a 34 llamadas. Esa ablación mide el efecto total de la carga bajo la misma política, no un grafo ejecutado idéntico. En la pareja jerárquica, ambos hicieron 96 llamadas y cargar diez skills elevó tokens un 89.3 %, sin aumentar el total de aciertos normalizados (22/24 en ambos).

Para este contrato estricto, SAGE es una alternativa adaptativa de consumo bajo frente a los cuatro equipos fijos. Si el sistema admite normalización del envoltorio SQL y prioriza el coste, los datos favorecen la referencia individual; si prioriza el cumplimiento normalizado observado, favorecen al reflexivo frente al secuencial por su menor consumo y tiempo medios. Esta decisión condicionada es más fiel a la evidencia que proclamar una arquitectura universalmente ganadora. La propuesta evaluada conserva una vía sin subagentes y utiliza redundancia solo en una parte del flujo. No incluye navegación, terminal, herramientas externas, memoria persistente, control de escrituras ni permisos de producción. Para una implementación real, el adaptador de contratos debe reemplazarse o ampliarse con recuperación de skills, medición de sus errores y límites de autoridad por rol.

La decisión de arquitectura debe separar suficiencia del resolutor, detección de riesgo, entrega de instrucciones y coste de transferencia. Un integrador fijo consume incluso cuando los trabajadores coinciden; una salida anticipada puede ahorrar ese paso, pero también aceptar un error compartido. Más roles o una etiqueta de skill distinta no crean independencia estadística entre respuestas del mismo modelo.

7. Validez y reproducibilidad

Las tareas finales ya se conocían al diseñar SAGE: esto es una réplica emparejada informada por desarrollo, no una prueba a ciegas de generalización. Las semillas de desarrollo distintas no eliminan el conocimiento de las familias ni de los errores del estudio anterior. La muestra es sintética y pequeña, con un solo modelo, dos repeticiones y contextos cortos; hay efectos techo y posibles fluctuaciones de proveedor o red. No se mide RAM, éxito en proyectos completos ni presupuesto igualado por respuesta correcta.

El paquete contiene las dos tandas separadas, los intentos de desarrollo, decisión de política, código congelado, protocolos, tareas, referencias del evaluador, trazas visibles por llamada, CSV, XLSX y fuentes LaTeX bilingües. No contiene credenciales ni razonamiento privado. Las claves se cargaron de un archivo local autorizado, ajeno al paquete público. Las cifras históricas no sustituyen controles de la réplica ni se mezclan con ellos para estimar latencia.

Apéndice: resultados históricos v1

La tanda anterior incluyó 144 ejecuciones, 416 llamadas y 568,283 tokens, conservados sin cambios. Su normalización SQL fue post hoc; en v2 se predefinió como secundaria. No se mezclan tandas.

Configuración	Estricto	SQL norm.	Tokens	s/tarea	Llamadas
Agente único	20/24	23/24	1,143	4.72	24
Secuencial	20/24	24/24	3,410	13.72	72
Paralela	19/24	23/24	3,520	8.14	72
Jerárquica	20/24	24/24	4,373	11.19	95
Reflexiva	20/24	24/24	2,714	9.77	58
Jerárquica + 10 skills	21/24	24/24	8,518	11.93	95

Cómo reproducir y auditar los materiales

El archivo público incluye código, datos y fuentes editables de ambas versiones. Descomprimirlo y trabajar desde su raíz, con las dependencias de `requirements.txt` instaladas en un entorno aislado. Para verificar y regenerar los resultados de la tanda conservada, sin llamar a la API:

```
python3 -m unittest discover -s benchmark -v
python3 -m unittest discover -s extension -v
python3 extension/analyze.py
python3 extension/write_results.py
python3 extension/write_web.py
```

Las pruebas offline no generan puntuaciones del modelo. El analizador comprueba las 216 ejecuciones, identidad de tareas, orden, copias congeladas, instrucciones cargadas y consumo por llamada. El generador escribe las secciones numéricas; las conclusiones editoriales se conservan aparte. Compilar dos veces cada `article.tex` desde `paper-v2/es` y `paper-v2/en` con pdfLaTeX.

Una nueva réplica real requiere una clave autorizada y tiene coste de API. Utilizar siempre otra carpeta de salida y seguir `extension/README.md`; no sobrescribir la decisión de política ni las trazas publicadas. El modelo es un alias: una réplica posterior no garantiza respuestas o tiempos idénticos.

La integridad del paquete v2 se comprueba con `ARTIFACTS-v2.sha256`; los archivos descargables tienen además un manifiesto `SHA256SUMS.txt`. Las trazas registran datos públicos sintéticos, respuestas visibles y uso informado, nunca encabezados de autorización.

Referencias

1. Siddhant Kulkarni y Yukta Kulkarni. *Benchmarking Multi-Agent LLM Architectures for Financial Document Processing: A Comparative Study of Orchestration Patterns, Cost-Accuracy Tradeoffs and Production Scaling Strategies*. 2603.22651v1. Primer envío: 2026-03-24. <https://arxiv.org/html/2603.22651v1>
2. Ziqi Wang, Yuhao Yang, Zhiwei Ling, Wenzhuo Qian y Hailiang Zhao. *When Does Hierarchy Help? Benchmarking Agent Coordination in Event-Driven Industrial Scheduling*. 2605.13172v1. Primer envío: 2026-05-13. <https://arxiv.org/html/2605.13172v1>
3. Zedong Yu et al.. *Beyond Sequential Interaction: Benchmarking Parallel Execution and Coordination for GUI Agents*. 2607.22689v1. Primer envío: 2026-07-17. <https://arxiv.org/html/2607.22689v1>
4. Zhenzhen Ren et al.. *OrchBench: Evaluating Multi-Agent Orchestration Plans in Isolation via Deterministic Simulation*. 2607.25656v1. Primer envío: 2026-07-28. <https://arxiv.org/html/2607.25656v1>
5. Abhijnan Nath, Hannah VanderHoeven y Nikhil Krishnaswamy. *CRAFT: Grounded Multi-Agent Coordination Under Partial Information*. 2603.25268v2. Primer envío: 2026-03-26. <https://arxiv.org/html/2603.25268v2>
6. Wasu Top Piriyaikulij, Rachel Lawrence, Alicia Curth, Sushrut Karmalkar y Niranjani Prasad. *Subagents vs Agent Skills: Executing Reusable Knowledge for Long-Horizon Agentic Tasks*. 2609.09233v1. Primer envío: 2026-09-07. <https://arxiv.org/html/2609.09233v1>
7. Aimin Zhang, Jiajing Guo, Fuwei Jia, Chen Lv, Boyu Wang y Fangzheng Li. *EvoAgent: An Evolvable Agent Framework with Skill Learning and Multi-Agent Delegation*. 2604.20133v3. Primer envío: 2026-04-22. <https://arxiv.org/html/2604.20133v3>
8. Xinyu Zhang et al.. *Swarm Skills: A Portable, Self-Evolving Multi-Agent System Specification for Coordination Engineering*. 2605.10052v2. Primer envío: 2026-05-11. <https://arxiv.org/html/2605.10052v2>
9. Renjun Xu y Yang Yan. *Agent Skills for Large Language Models: Architecture, Acquisition, Security, and the Path Forward*. 2602.12430v4. Primer envío: 2026-02-12. <https://arxiv.org/html/2602.12430v4>
10. OpenAI. *GPT-5.6 Luna Model*. Consultado el 2026-09-13. <https://developers.openai.com/api/docs/models/gpt-5.6-luna>
11. Zhiqing Cui et al. *Uno-Orchestra: Parsimonious Agent Routing via Selective Delegation*. 2605.05007v1. Primer envío 2026-05-06. <https://arxiv.org/html/2605.05007v1>
12. Songyuan Li, Ahmed M. Abdelmoniem y Shiqiang Wang. *ProgRouter: Online Progress-Guided Orchestration for Multi-Agent LLM Workflows under Quality-Cost Tradeoffs*. 2608.25992v1. Primer envío 2026-08-26. <https://arxiv.org/html/2608.25992v1>
13. Tianxing Wang et al. *TROVE: Adaptive Agent Skill Orchestration via Trace-Grounded Route Validation and Editing*. 2609.05019v1. Primer envío 2026-09-04. <https://arxiv.org/html/2609.05019v1>